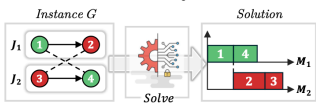


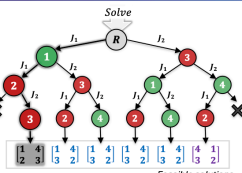
Job Shop Scheduling Problem

The Job Shop asks to schedule jobs across multiple machines. Each job comprises a sequence of operations indicating the order in which the job must be processed on the machines. The goal is to find start times for operations that respect job precedences and minimize an objective function.



Constructive Algorithms

Build feasible solutions for combinatorial problems as a *sequence of decisions*. At each decision, a rule selects the next entity – a job with a ready operation in Job Shop – to include in the solution.



Motivations

Constructive algorithms, such as dispatching rules, rely on rigid, often hand-crafted rules, which limits their ability to consistently produce high-quality solutions.

↳ **Learn better rules** from data with deep learning.

Supervised learning
Generate (near)-optimal solutions and learn rules for constructing them.

Obtaining optimal solutions is **expensive** and introduces **multi-modality issues** (i.e. construction may require completely different rules).

Reinforcement learning
Define a reward signal for the problem and learn a rule to maximize it.

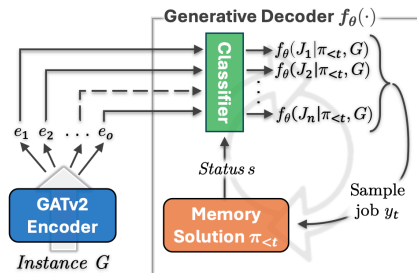
Reward signals are often **sparse**, and the training is **complex and sensitive to hyperparameters**.



Intuition. Use **semi-** and **self-supervised** learning to avoid issues of supervised and reinforcement learning methodologies.

Assumptions of our methodology:

- Identify a combinatorial problem and define its **objective function**. For instance, use Job Shop Scheduling and construct solutions minimizing the total completion time (*makespan*).
- Design a **generative model** $f_\theta(\cdot)$, such as a Pointer Network or a Transformer, that will learn the rule. At each decision t , $f_\theta(\cdot)$ outputs the probability of selecting an entity (e.g., a job in Job Shop) from the representation of the instance G and the partial solution $\pi_{<t}$ constructed up to t . →



Update Model

Adjust the parameters θ of the model by *minimizing the cross-entropy loss* of all decisions y_t .

$$L(\pi^*) = -\frac{1}{o} \sum_{t=1}^o \log f_\theta(y_t | \pi_{<t}^*, G)$$

Select Pseudo-Label

Select the best sampled solution π^* according to the problem objective as a *pseudo-label*.

Generate Instance

Arbitrarily generate an *instance G* for a combinatorial problem or sample it from a dataset.

Construct Solutions

Create β (parallel) solutions by sampling *autoregressively* from the generative model $f_\theta(\cdot)$.

$$\pi = \prod_{t=1}^o f_\theta(\cdot | \pi_{<t}, G)$$



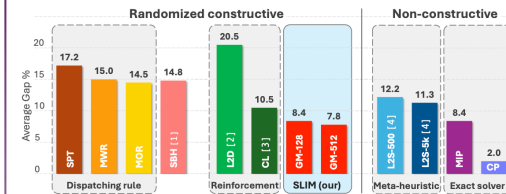
Paper



Code

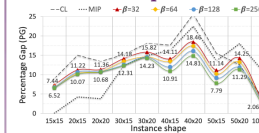
- J. Adams et al. The Shifting Bottleneck Procedure for Job Shop Scheduling. *Management Science*, 34, 1988.
- C. Zhang et al. Learning to dispatch for job shop scheduling via deep reinforcement learning. *Advances in Neural Information Processing Systems*, 33, 2020.
- Z. Iklasov et al. On the Study of Curriculum Learning for Inferring Dispatching Policies on the Job Shop Scheduling. In *Proceedings of the International Joint Conference on Artificial Intelligence*, 2023.
- C. Zhang et al. Deep Reinforcement Learning Guided Improvement Heuristic for Job Shop Scheduling. In *International Conference on Learning Representations*, 2024.

Experiment & Analysis



Key Message. Compared to existing approaches, our methodology allows training better neural constructive algorithms, almost achieving the performance of more powerful non-constructive approaches.

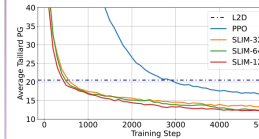
Effect of Sampled Solutions β on SLIM



Take Home

SLIM is effective even with a small number of sampled solutions β .

SLIM vs PPO in Job Shop Scheduling



Take Home

Within the training setup of [2], SLIM trains better models than Proximal Policy Optimization.

Self-Labeling the Traveling Salesman Problem

Take Home

Due to its assumptions, SLIM can address other combinatorial problems like the Traveling Salesman Problem.

